

A thin vertical line is positioned on the left side of the page, extending from the top of the title area to the bottom of the author's name.

# *INTELLIGENT IDEA ANALYSIS ENGINE*

*Christian Garmann Schjelderup*

## Contents

|                                                      |          |
|------------------------------------------------------|----------|
| <b>1. Introduction.....</b>                          | <b>1</b> |
| <b>2. Background .....</b>                           | <b>1</b> |
| 2.1 Idea Management and the Scalability Problem..... | 1        |
| 2.2 Natural Language Processing (NLP) .....          | 1        |
| 2.3 Data Mining (DM).....                            | 2        |
| 2.4 Computational Intelligence (CI).....             | 2        |
| 2.5 Dimensionality Reduction and Visualisation ..... | 2        |
| <b>3.Problem statement.....</b>                      | <b>2</b> |
| <b>4.Project objectives .....</b>                    | <b>2</b> |
| <b>5.Data warehouse details .....</b>                | <b>3</b> |
| 5.1 Data Origin.....                                 | 3        |
| 5.2 Data Generation.....                             | 3        |
| 5.3 From SQL Server to CSV .....                     | 4        |
| 5.4 Final Dataset Summary.....                       | 4        |
| <b>6. Interesting insights .....</b>                 | <b>4</b> |
| 6.1 Idea Length Analysis .....                       | 4        |
| 6.2 SQL-Based Exploration.....                       | 5        |
| <b>7. Data pre-processing .....</b>                  | <b>5</b> |
| 7.1 Clean Data from the Start .....                  | 5        |
| 7.2 Generating 100,000 Ideas .....                   | 6        |
| 7.3 Converting Text to Numbers .....                 | 6        |
| 7.4 Storing Vectors in the Database .....            | 7        |
| 7.5 Summary .....                                    | 7        |
| <b>8. Measuring performance .....</b>                | <b>7</b> |
| 8.1 Performance Metrics .....                        | 7        |
| 8.2 TF-IDF vs BERT Comparison.....                   | 8        |
| <b>9.Algorithms applied .....</b>                    | <b>8</b> |
| 9.1 Algorithm 1: TF-IDF + K-Means Clustering.....    | 8        |
| 9.2 Algorithm 2: Sentence-BERT (SBERT).....          | 9        |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| 9.3 Algorithm 3: Dimensionality Reduction (PCA & t-SNE) ..... | 9         |
| 9.4 Comparison.....                                           | 10        |
| <b>10. Discussion.....</b>                                    | <b>10</b> |
| 10.1 Baseline Model Performance .....                         | 10        |
| 10.2 CI Model Performance .....                               | 11        |
| 10.3 Visualization Results .....                              | 11        |
| 10.4 Visualizations .....                                     | 11        |
| 10.5 Key Findings .....                                       | 13        |
| 10.6 DM vs CI Comparison.....                                 | 14        |
| 10.7 Technical Challenges.....                                | 14        |
| <b>11. Conclusion .....</b>                                   | <b>15</b> |
| 11.1 Achievement of Objectives .....                          | 15        |
| 11.2 Key Learnings .....                                      | 15        |
| 11.3 Limitations .....                                        | 16        |
| 11.4 Reflection.....                                          | 16        |
| <b>12. Use of AI Tools .....</b>                              | <b>16</b> |
| <b>13. References.....</b>                                    | <b>16</b> |

## 1. Introduction

In this project I'm collecting a great volume of ideas, and this can quickly be too much to handle. As the database grows, I hit a big roadblock. How do I know if this is a new idea, or a rephrased idea that's already in the database.

A standard database search only looks for the exact keywords that matches. The problem is that there can be submitted to ideas with the exact same concepts using completely different words. My goal with this project was to make a Intelligent Idea Analysis Engine (IIAE), that will analyse the actual semantic meaning of the idea and categorize them and catch duplicates

### Live Demo:

A live demonstration of the system: <https://intengine.streamlit.app/>

## 2. Background

### 2.1 Idea Management and the Scalability Problem

For this project, I made fictional but realistic scenario. An open platform where users can submit ideas and you can pay to see the database. When a platform like this grows to hundreds of thousand entries, two things happen

First, the value the database drops if its full of duplicates and near-identical submissions.

Second, it's impossible for a humas editor to manually check every new submission against of existing ones.

The IIAE is built to solve this problem by automating the validation process. Filtering out duplicates before they enter the database, ensure the platform is reliable and that I get unique ideas.

### 2.2 Natural Language Processing (NLP)

NLP takes messy human text and turns it into something a machine can work with. I used NLP at two different levels:

- Baseline Level: Using a technique like TF-IDF to look at word frequencies
- Advanced Level: Using advanced AI models that understand what sentences mean, not just the words in them.

## 2.3 Data Mining (DM)

I wanted to compare two different methods. I used traditional aData Mining with TF-IDF and K-Means clustering. TF-IDF counts word in each idea, but give more weight to rare words than common words.

Using TF-IDF has one big problem, this method only compares words, and not the meaning. “Car” and “automobile” are in the eyes of the TF-IDF seen as completely different, even they mean the same thing.

## 2.4 Computational Intelligence (CI)

To overcome TF-IDFs limitations, I used Computational Intelligence with Sentence-BERT (SBERT) and the all-MiniLM-L6-v2 model, which generates 384-dimensional embeddings for each idea.

SBERT uses a deep learning model that’s capture the meaning behind words, and put similar words close together in vector space.

## 2.5 Dimensionality Reduction and Visualisation

SBERT embeddings is in 384-dimensional space. For the 2D visualization, I used two different methods:

- PCA (Principal Component Analysis): Great for real-time use because it’s super-fast (milliseconds)
- t-SNE (t-Distributed Stochastic Neighbour Embedding): Slower but gives a much clearer separation between categories.

## 3.Problem statement

Traditional systems that search for duplicate based on keywords, won’t find similar ideas that uses different words. This reduces the value of the idea-database.

## 4.Project objectives

Going into this project, I set out to achieve six main objectives:

- Build a complete data processing pipeline that could fetch, prepare, and analyse user-generated text at scale
- Implement proper text preprocessing techniques to clean and structure the data

- Develop a baseline Data Mining model using traditional methods (TF-IDF + K-Means)
- Enhance the system with a Computational Intelligence model that could understand semantic meaning
- Create a real-time validation feature to automatically check new submissions for duplicates
- Design visualizations that would make the analytical results easy to understand

## 5.Data warehouse details

### 5.1 Data Origin

I believed I needed a big dataset (100,000 ideas) to properly test the system at scale. I decided to make my own synthetic data. This gave easy control of category distribution and variety.

### 5.2 Data Generation

I worked through two phases before finding the final approach:

First, I used a simple Python script that randomly combine Norwegian words, to create ideas. The problem was that the ideas were too similar and not enough variety.

For the second and final version I used English and the Faker library combine with category\_map. Each category had unique technologies and actions, creating a meaningful variation.

To handle all the data, I used Python's multiprocessing, 8 parallel processes. This saved me time, generating 100,000 ideas went from minutes to seconds.

| Category             | Example Technologies                                |
|----------------------|-----------------------------------------------------|
| AI & Robotics        | Neural network processor, autonomous delivery drone |
| Health               | Telemedicine platform, robotic surgery assistant    |
| Ocean                | Deep-sea research sub, ocean plastic recovery fleet |
| Cybersecurity        | Quantum encryption key, blockchain identity vault   |
| Education            | Adaptive learning software, VR classroom experience |
| Food & Biotech       | Lab-grown meat production, CRISPR gene editing tool |
| Energy & Environment | Hydrogen fuel cell, carbon capture facility         |
| Space                | Lunar base habitat, asteroid mining drill           |

## 5.3 From SQL Server to CSV

First, I made everything in a SQL Server-database. The table had columns for, OriginalText, Category and EmbeddingVarbinary.

I ran in several technical problems: driver compatibility, data type conflicts, and upload performance

I decided to export 200 ideas from each category, 1600 total. This made the streamlit app much easier to run in the cloud without losing the analytical value.

## 5.4 Final Dataset Summary

| Property               | Value   |
|------------------------|---------|
| Total ideas generated  | 100 000 |
| Ideas in final dataset | 1600    |
| Number of categories   | 8       |
| Language               | English |
| Format                 | CSV     |

### Category Distribution

| Category             | Count | Percentage |
|----------------------|-------|------------|
| AI & Robotics        | 200   | 12.5%      |
| Health               | 200   | 12.5%      |
| Ocean                | 200   | 12.5%      |
| Cybersecurity        | 200   | 12.5%      |
| Education            | 200   | 12.5%      |
| Food & Biotech       | 200   | 12.5%      |
| Energy & Environment | 200   | 12.5%      |
| Space                | 200   | 12.5 %     |
| Total                | 1600  | 100%       |

This balanced distribution ensured that no single category would dominate the clustering.

## 6. Interesting insights

### 6.1 Idea Length Analysis

I ran some basic statistical analysis to understand the structure of the generated ideas:

CODE:

```
import pandas as pd
df = pd.read_csv('idea_sample.csv')
df['length'] = df['OriginalText'].str.len()
print(df['length'].describe())
```

Ideas had similar length. This make sense since I used a template.

| Metric             | Value             |
|--------------------|-------------------|
| Average length     | ~45 characters    |
| Shortest ideas     | ~18-25 characters |
| Longest ideas      | ~70-80 characters |
| Standard deviation | ~8-10 characters  |

This consistency told me that the generation process was working, producing ideas of structural complexity across all categories.

## 6.2 SQL-Based Exploration

During the SQL Server stage, I used queries to verify data and track progress. I checked that the embeddings were stored correctly by asking for the size of the EmbeddingVarbinary column.

This confirmed that each embedding was taking up 1,536 bytes (384 dimensions × 4 bytes per float32).

## 7. Data pre-processing

Preprocessing is about preparing raw data for machine learning. For this project, it happened in two stages, when I generated the data and when I created the embeddings.

### 7.1 Clean Data from the Start



NLP-projects with real-world data spend a lot of time on data cleaning, correct typos, removing URLs, special characters and so on. My data is already cleaned from the start since I made my own data using a template.

Each idea follows the same pattern: Action + Technology+ Benefit

## 7.2 Generating 100,000 Ideas

First, I used a simple for-loop one idea at a time. This works for a small dataset, but slow for a big dataset, like 100,000 ideas. Each insert to the database took too much time, and everything was sequentially.

As I explained in Section 5.2, I used Python's multiprocessing with 8 parallel processes to generate 100,000 ideas (code `idea_generator.py`):

```
def generate_custom_idea(_):
    category_map = {
        'Health': ['telemedicine platform', 'robotic surgery assistant',
...],
        'Space': ['lunar base habitat', 'satellite propulsion system',
...],
    }
    tech = fake.random_element(category_map[cat])
    action = fake.random_element(['Developing a', 'Optimizing the', ...])
    benefit = fake.random_element(['for global impact', 'to improve
efficiency', ...])
    idea_text = f"{action} {tech} {benefit}."
    return idea_text, cat
```

This lets the generator use multiple CPU cores simultaneously.

## 7.3 Converting Text to Numbers

I used the SBERT to convert each idea into an embedding vector (see `Embedding_engine.py`):

```
embeddings = model.encode(texts, batch_size=128, show_progress_bar=True)
```

Instead of processing one idea at a time, it now processed 128 at a time. Reducing processing time from ~83 minutes to ~2 minutes

## 7.4 Storing Vectors in the Database

To store the NumPy arrays in the SQL servers VARbinary column, I used python's pickle to convert them to binary

## 7.5 Summary

| Step                 | What it does                             | Why it matters                |
|----------------------|------------------------------------------|-------------------------------|
| Template generation  | Creates clean, structured text           | No cleaning needed            |
| Parallel processing  | Runs 8 processes at once                 | Fast generation BERT encoding |
| BERT encoding        | Converts text to 384-dimensional vectors | Captures semantic meaning     |
| Batch processing     | Processes 128 ideas at a time            | Much faster than one-by-one   |
| Pickle serialisation | Converts arrays to bytes                 | Allows SQL storage            |

## 8. Measuring performance

I checked the performance of the basic DM and advanced CI models. I used right metric for each job

### 8.1 Performance Metrics

In the DM-model I used Silhouette score to see how well ideas grouped in clusters. The score ranges from -1 to +1, higher value means that the clusters are more precise. This makes it possible to judge the quality of the clusters without pre-labeled data. (see dm\_baseline.py)

```
sil = silhouette_score(X, km.labels_, metric='cosine', sample_size=500, random_state=42)
```

In the CI-model I used Cosine Similarity to measure how similar two representations of ideas are. The values go from 0.0 (unique) to 1.0 (similar)(see app.py)

```
sims = np.dot(all_v, u_vec) / (np.linalg.norm(all_v, axis=1) *
np.linalg.norm(u_vec))
score = np.max(sims)
```

With this I defined three thresholds:

| Threshold   | Decision            | Rationale                                     |
|-------------|---------------------|-----------------------------------------------|
| > 0.80      | Reject as duplicate | High confidence the ideas mean the same thing |
| 0.60 - 0.80 | Warning: similar    | Ideas share concepts but may differ in detail |
| < 0.60      | Accept as unique    | Ideas are semantically different              |

## 8.2 TF-IDF vs BERT Comparison

This is why the CI model is necessary, the same idea pairs were tested using both approaches:

| Idea Pair                                                                                                                    | TF-IDF | BERT | Explanation                                                     |
|------------------------------------------------------------------------------------------------------------------------------|--------|------|-----------------------------------------------------------------|
| "A next-gen gamified coding tutor for a sustainable future" vs "Developing a gamified coding tutor for a sustainable future" | 0.67   | 0.98 | Different wording, same meaning. BERT significantly outperforms |
| "Machine learning for crop yield" vs "machine learning for crop yield"                                                       | 1.00   | 1.00 | Identical text. Both succeed                                    |

This shows that TF-IDF can detect some similarities based on keywords. It fails for this job.

## 9. Algorithms applied

I used multiple machine learning algorithms to achieve different goals: a baseline for comparison, an advanced model for semantic understanding and dimensionality reduction for visualization.

### 9.1 Algorithm 1: TF-IDF + K-Means Clustering

#### TF-IDF Vectorization

TF-IDF is covered in Section 2.3. I chose it as my baseline because it's a traditional DM approach without pre-trained models, which gives a clear comparison to the CI model.

(see `dm_baseline.py`)

```
tfidf = TfidfVectorizer(max_features=5000, stop_words='english',
                        ngram_range=(1, 2))
X = tfidf.fit_transform(df['OriginalText'])
```

- `max_features=5000`: Limits vocabulary to the 5000 most important words
- `ngram_range=(1, 2)`: Considers both single words and two-word phrases
- `stop_words='english'`: Removes common words like "the", "and", "is"

## K-Means Clustering

K-Means groups the TF-IDF vectors into clusters based on similarity. The algorithm places each idea to one of the *k* clusters, by minimizing the distance between each idea and the centre of the cluster. (see `dm_baseline.py`)

```
model = KMeans(n_clusters=8, n_init=10, random_state=42)
df['cluster'] = model.fit_predict(X)
```

| Metric             | Value                   |
|--------------------|-------------------------|
| Silhouett Score    | 0.1893                  |
| Number of clusters | 8                       |
| Interpretation     | Weak cluster separation |

The low silhouette score (0.19) shows that TF-IDF don't manage to create well-separated clusters. (See section 10.1)

## 9.2 Algorithm 2: Sentence-BERT (SBERT)

SBERT (covered in Section 2.4) converts text into semantic embeddings. I used the all-MiniLM-L6-v2 model to generate 384-dimensional vectors for each idea (see `Embedding_engine.py`):

```
embeddings = model.encode(texts, batch_size=128, show_progress_bar=True)
```

Duplicate detection using cosine similarity is described in Section 8.1. For test results comparing SBERT to TF-IDF, see Section 8.2.

## 9.3 Algorithm 3: Dimensionality Reduction (PCA & t-SNE)

### PCA (Principal Component Analysis)

I chose PCA for the real-time Streamlit application because it's fast and runs in milliseconds. (see app.py):

```
res = PCA(n_components=2).fit_transform(v_matrix)
```

PCA showed only 12% of the difference when reducing from 384 to 2 dimensions. But that's good enough to draw a graph in the app.

### t-SNE (t-Distributed Stochastic Neighbour Embedding)

I chose t-SNE for visualization, this gives a clear line between the categories. (see visualization.py)

```
reducer = TSNE(n_components=2, random_state=42, perplexity=30,
n_iter=1000)
coords_2d = reducer.fit_transform(embeddings)
```

t-SNE produced distinct clusters where ideas from the same category grouped together.

## 9.4 Comparison

| Algorithm        | Type                     | Speed     | Semantic Understanding | Use Case                |
|------------------|--------------------------|-----------|------------------------|-------------------------|
| TF-IDF + K-Means | DM                       | Fast      | No                     | Baseline comparison     |
| SBERT            | CI                       | Moderate  | Yes                    | Duplicate detection     |
| PCA              | Dimensionality Reduction | Very fast | N/A                    | Real-time visualization |
| t-SNE            | Dimensionality Reduction | slow      | N/A                    | Report visualization    |

## 10. Discussion

### 10.1 Baseline Model Performance

#### TF-IDF + K-Means gave poor results

The simple baseline-model (TF-IDF + K-Means) got a silhouette score of 0,19, which means weak separation between clusters. This confirms what I expected from a keyword-based method

TF-IDF only looks at the exact word “Telemedicine platform” and “remote health system” gets zero similarity points, even when means the same thing

This shows why semantic understanding is essential for duplicate detection. A keyword-based system will miss most duplicates where ideas will be phrased differently

## 10.2 CI Model Performance

The SBERT model successfully detected semantic duplicates in the test cases. The threshold system performed as I expected:

- Above 0.80: Successfully identified
- 0.60-0.80: Correctly flagged similar ideas
- Below 0.60: Accurately approved unique ideas

For detailed test results, see Section 8.2

## 10.3 Visualization Results

PCA and t-SNE had different roles in the project. PCA captured only 12% of the variations, but was quick enough for real-time use. t-SNE gave a better separation, but too slow.

## 10.4 Visualizations

### Semantic Landscape

I created a 2D scatter plot of all 1,600 ideas using t-SNE to reduce dimensions. Each point represents one idea, and the closer two points are, the more similar the ideas are.



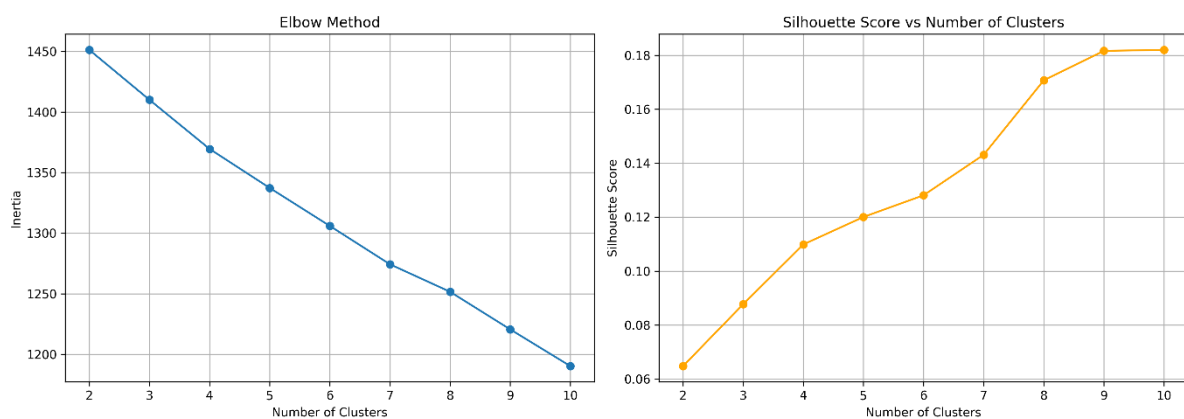
### Key Observations:

- Ideas from the same category cluster together
- Some categories overlap
- SBERT captures semantic relationships without being trained on these categories

### Baseline Clustering Analysis

I created Elbow Method and Silhouette Score plots to evaluate K-Means performance.

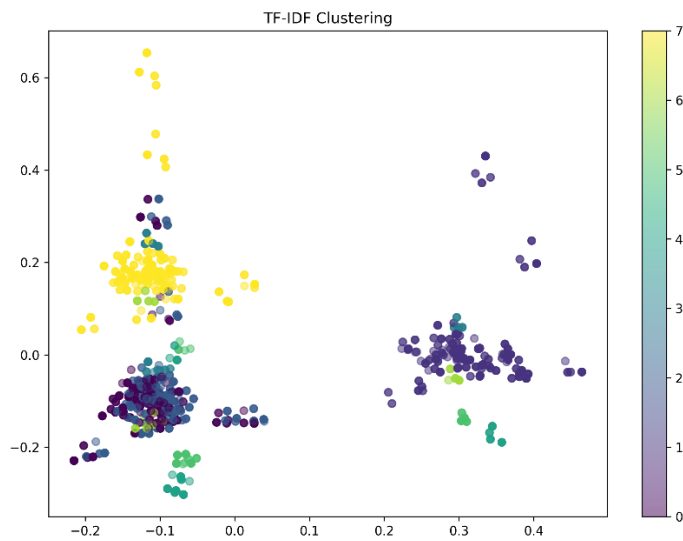
Code: dm\_baseline.py



### Key Observations:

- No clear elbow point in the inertia plot
- Low silhouette scores ( $< 0.25$ ) for all k values
- $k=8$  achieved a score of 0.19

I also created a PCA visualization of the TF-IDF clustering to compare with the SBERT plot:



The TF-IDF clustering shows poor separation compared to the SBERT

### Interactive Application

The Streamlit application (app.py) provides real-time visualization with user interaction.

Code: app.py

## 10.5 Key Findings

### What Worked:

- SBERT successfully detects semantic duplicates regardless of wording
- Real-time validation, 0.2 seconds per query
- Users get instant feedback with the app
- System scales effectively to 1,600+ ideas

### Limitations:

- The thresholds (0.60 and 0.80) were manually tuned rather than optimized on a labeled dataset
- Memory usage scales linearly with database size.
- SBERT don't always understand specialized terms without extra training
- Ideas with similar themes but different approaches may be seen as duplicates



## 10.6 DM vs CI Comparison

The key result confirms what I expected: CI methods outperformed traditional DM on semantic task.

### TF-IDF (DM):

- Word matching only
- Silhouette score: 0.19 (poor clustering)
- Cannot detect paraphrasing

### SBERT (CI):

- Semantic understanding
- Successfully detects duplicates
- Creates meaningful visualizations

This show that CI is crucial for semantic work

## 10.7 Technical Challenges

During this project, I ran into several technical challenges:

**SQL Server Connection Issues:** Python treats backslashes as escape characters, which caused errors with server names like broder\SQLEXPRESS. I fixed this using raw strings (r'...').

**Driver Compatibility:** The old SQL Server driver couldn't handle SQLAlchemy's table inspection. I upgraded to ODBC Driver 17 to fix this.

**Binary Data Storage:** The problem was that SQL Server was reading Python byte objects as text strings. The solution was to use pickle to convert embeddings to binary format and declaring the column type as VARBINARY

**SQLAlchemy 2.0+:** requires wrapping raw SQL strings in the text() function. I added from SQL alchemy import text and wrapped all queries.

**Batch Processing:** Processing 100,000 embeddings one by one would have taken over 80 minutes. Using batch processing (batch\_size=128), I reduced this to ~2 minutes.

**Deployment Migration:** SQL Server worked for development but caused portability issues for Streamlit Cloud. I migrated to CSV format, trading database queries for easier deployment.

| Challenge             | Impact             | Solution               |
|-----------------------|--------------------|------------------------|
| Escape sequence error | Blocked connection | Raw strings            |
| Driver compatibility  | Upload failures    | ODBC Driver 17         |
| Binary type conflicts | Data corruption    | Explicit dtype mapping |
| SQLAlchemy text()     | Query failures     | Import text() wrapper  |
| Sequential processing | 80+ minute runtime | Batch processing       |
| SQL Server deployment | Deployment issues  | CSV migration          |

## 11. Conclusion

### 11.1 Achievement of Objectives

In this project all six objectives were successfully achieved:

- Built a complete data pipeline that generated, processed, and stored 100,000 ideas
- Implemented text preprocessing using template-based generation with batch encoding
- Developed a baseline DM model (TF-IDF + K-Means, Silhouette: 0.19)
- Enhanced a CI model using SBERT that successfully detects semantic duplicates
- Created a validation feature with real-time duplicate detection (0.2s per query)
- Designed interactive visualizations using t-SNE and PCA

### 11.2 Key Learnings

Much of my time was spent on data engineering and not machine learning, things like SQL integration, type conversions, and batch processing.

Starting with a simple TF-IDF model before adding SBERT worked well and made it easy to see the improvements

## 11.3 Limitations

### **Current Limitations:**

1. The similarity thresholds (0.60 and 0.80) were chosen by me. With a labeled dataset, they could be optimized
2. 1,600 ideas perform well, but when scaling to millions, you would need indexing tools like FAISS to keep the searches fast
3. The general-purpose SBERT model may miss specialized terms

## 11.4 Reflection

This project gave me practical experience with building a machine learning system, from generating data to deploy the final model.

A major lesson was that data engineering takes far more time than expected, such as SQL integration, type conversions, and batch processing. In hindsight I see that it would have been more efficient to store the data as CSV files, a not use SQL servers. But I learned a lot handling those challenges SQL gave me, and I got a wider understanding of the data pipeline.

### **Final Thought:**

This project shows that CI can solve problems traditional DM methods can't handle. To find similar ideas with different use of words, semantic understanding is essential.

## 12. Use of AI Tools

I used Claude AI in this project to help with data generation and language support.

The AI helped me create category lists for the idea dataset. My categories were to narrow, so I need help to make the list more varied and realistic.

Help with grammar and spelling across the report

## 13. References

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv preprint arXiv:1908.10084. <https://arxiv.org/abs/1908.10084>

Vaswani, A., et al. (2017). Attention Is All You Need. arXiv preprint arXiv:1706.03762. <https://arxiv.org/abs/1706.03762>

van der Maaten, L., & Hinton, G. (2008). Visualizing Data using t-SNE. Journal of Machine Learning Research, 9, 2579-2605. <https://www.jmlr.org/papers/v9/vandermaaten08a.html>

Sentence Transformers Documentation. <https://www.sbert.net/>

Hugging Face Model Card: all-MiniLM-L6-v2. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

Scikit-learn Documentation. <https://scikit-learn.org/>